Bloxorz - Kết quả UCS và rà soát cost

Phan Gia Phúc - MSSV 24120503. P03, ngày 31/10/2026. Người nhận: Nam ghép báo cáo; Quân đối chiếu nguồn; Thiện rà quan hệ cost và heuristic.

1. Căn cứ và kết quả tổng hợp

Phân tích dùng 25 màn × 4 thuật toán × 3 lần lặp = 300 lượt trong bộ dữ liệu của Quân. Cả 300 lượt có trạng thái SOLVED; không có NO_SOLUTION, TIMEOUT, ERROR hoặc INVALID_PATH. Mỗi cặp màn/thuật toán có n=3 mẫu. Thời gian, bộ nhớ, node, độ dài và cost trong tài liệu là trung vị của từng cặp, theo đúng `summary.csv`. [1]

Checkout dùng để rà code và tài liệu là `ce04ee45ead1702365646e41b4cbad92b8818304`. CSV/JSON trùng các hash đã dùng trong P05. Metadata của lượt đo không ghi commit gốc; commit trên xác định checkout phân tích. `ucs-results-data.json` lưu hash nguồn, kết quả đối chiếu và các phép so sánh; `ucs-results-selected.csv` lưu 21 dòng của bảy màn đại diện. Phân tích không chạy lại solver hoặc replay. [1][2]

Trong bảng dưới, “thấp/bằng/cao” so sánh giá trị trung vị của thuật toán đứng trước với thuật toán đứng sau trên từng màn. Mỗi hàng có tổng 25 màn; đây không phải tỷ lệ trên 75 lượt của một solver và không phải kiểm định ý nghĩa thống kê.

So sánh	Chỉ số	Thấp	Bằng	Cao
UCS / BFS	path_cost	3	22	0
UCS / BFS	Số hành động	0	23	2
UCS / BFS	Thời gian	1	0	24
UCS / BFS	Đỉnh cấp phát Python	0	0	25
UCS / BFS	Node mở rộng	0	1	24
A* / UCS	path_cost	0	25	0
A* / UCS	Số hành động	0	25	0
A* / UCS	Thời gian	20	0	5
A* / UCS	Đỉnh cấp phát Python	20	0	5
A* / UCS	Node mở rộng	24	1	0

UCS giảm cost so với BFS ở L16, L24 và L25. A* trả cùng cost và số hành động với UCS trên cả 25 màn đã đo. A* mở ít node hơn UCS trên 24 màn, bằng nhau tại L11; lợi thế về node không đồng nghĩa luôn nhanh hơn hoặc dùng ít bộ nhớ hơn. [1][2]

Hàm mục tiêu được giữ nguyên theo P01: chi phí một cạnh hợp lệ bằng `1 + 0.5 × k`, với k là số ô fragile mà toàn bộ khối chiếm ở trạng thái đến sau xử lý luật. `path_cost` là tổng chi phí các cạnh. BFS/DFS trả `total_cost` gốc bằng số hành động; UCS/A* trả cost có trọng số. So chéo bốn thuật toán bằng `path_cost` trong CSV; không lấy hai nghĩa của `total_cost` làm cùng một đại lượng. [3][4]

2. Chất lượng lời giải: số bước và cost

L16 là Split Maze, có 64 công tắc tách. L24 và L25 là Short or Cheap và bản quay hướng của màn này; mỗi màn có 28 ô fragile. Ba màn làm rõ hai cách cải thiện cost: giữ nguyên số hành động nhưng giảm tiếp xúc fragile, hoặc đi nhiều hành động hơn để giảm tổng cost. [1]

Màn	Solver	Hành động L	path_cost C	Lượt chiếm ô fragile F
L16	BFS	110	111.5	3
L16	UCS	110	110.5	1
L16	A*	110	110.5	1
L24	BFS	16	23.0	14
L24	UCS	20	20.0	0
L24	A*	20	20.0	0
L25	BFS	16	23.0	14
L25	UCS	20	20.0	0
L25	A*	20	20.0	0

Nguồn: `summary.csv`, khóa `(level_id, solver)` tương ứng; độ dài/cost trên ba lần lặp của từng cặp trong bảng đều giống nhau. F được suy ra từ công thức `C = L + 0.5 × F`, tức `F = 2 × (C - L)`. Đây là tổng số lượt ô fragile bị chiếm tại các trạng thái đến: một ô có thể bị đếm nhiều lần, một trạng thái có thể đóng góp hai ô. F không phải số ô phân biệt và không phải xác suất rơi. [1][2][3]

Ở L16, UCS giữ 110 hành động như BFS nhưng giảm cost 1.0, tương ứng giảm từ ba xuống một lượt chiếm ô fragile. Chỉ so độ dài sẽ bỏ qua khác biệt này.

Ở L24 và L25, UCS đi thêm bốn hành động, tăng 25% so với BFS, nhưng giảm cost từ 23.0 xuống 20.0, tức giảm 13.04%. Phần phạt của BFS là 7.0; UCS loại phần phạt này và trả thêm 4.0 chi phí hành động, nên tổng cost giảm 3.0. Đây là ví dụ thật cho việc đường ít bước nhất có thể không phải đường ít cost nhất. [1][2]

Với chi phí cạnh dương hiện tại và kiểm tra goal khi lấy khỏi min heap, UCS tối ưu tổng cost. BFS tối ưu số hành động vì mỗi cạnh tìm kiếm được coi là một bước; BFS không tối ưu cost có trọng số trong mọi màn. Khi toàn bộ cạnh có cost 1, hai mục tiêu trùng nhau, nhưng cách dừng tại goal vẫn có thể tạo số node và thời gian khác nhau. [3][4]

Hình P05 để đối chiếu: `charts/04_solution_length.png` và `charts/05_path_cost.png`, cụm L16-L20 và L21-L25. Hai hình dùng trục y log; dùng bảng số khi cần đọc mức chênh lệch tuyệt đối.

3. Hiệu năng tìm kiếm trên các màn đại diện

Bảng sau chọn L1 (cơ bản, không fragile), L4 (tách/ghép), L8 (ngoại lệ thời gian/bộ nhớ của A*), L12 (mê cung cầu) và L16 (64 công tắc tách). Mỗi ô có n=3 mẫu. Thời gian tính bằng ms; bộ nhớ là đỉnh cấp phát Python qua tracemalloc, đơn vị KiB; node đếm lượt sinh successor. Dấu phẩy phân tách hàng nghìn, dấu chấm là dấu thập phân. [1][4]

Màn	Solver	Thời gian (ms)	Bộ nhớ (KiB)	Node mở rộng
L1	BFS	2.639	16.73	35
L1	UCS	5.485	25.66	55
L1	A*	2.163	13.20	18
L4	BFS	357.309	543.01	1,789
L4	UCS	500.618	705.98	1,903
L4	A*	354.592	601.08	1,245
L8	BFS	17.205	66.20	201
L8	UCS	22.036	81.10	208
L8	A*	25.398	82.38	192
L12	BFS	75.837	321.86	928
L12	UCS	109.480	413.37	995
L12	A*	50.082	228.16	451
L16	BFS	21,128.143	15,344.75	67,472
L16	UCS	19,903.749	19,741.03	68,095
L16	A*	35,061.705	19,546.84	65,951

Nguồn: [1], các khóa `(1|4|8|12|16, bfs|ucs|astar)`; bản trích trong [2]. Hình P05 tương ứng: `01_search_time_ms`, `02_memory_kb` và `03_expanded_nodes`.

UCS so với BFS. UCS chậm hơn trên 24/25 màn và dùng nhiều bộ nhớ đỉnh hơn trên cả 25 màn; L16 là ngoại lệ về thời gian. Ngay tại L1 có cost đồng nhất, UCS mở 55 node so với 35 của BFS. BFS dừng khi sinh goal, còn UCS đợi lấy goal khỏi heap; heap và các bảng best_g/parent cũng tạo công việc khác deque của BFS. Không quy toàn bộ chênh lệch cho hàm cost. [1][4]

A* so với UCS. L12 cho thấy lợi ích rõ: A* giảm node từ 995 xuống 451 (54.67%), thời gian từ 109.480 xuống 50.082 ms (54.25%) và bộ nhớ từ 413.37 xuống 228.16 KiB (44.80%). Cả hai vẫn có 28 hành động và cost 28.0. [1][2]

Ngoại lệ cần giữ trong nhận xét. Tại L8, A* mở ít hơn 16 node (7.69%) nhưng chậm hơn 15.26% và dùng nhiều bộ nhớ hơn 1.58%. Tại L16, A* giảm 3.15% node nhưng tăng 76.16% thời gian so với UCS. Đây là so sánh mô tả của bộ ba lần lặp, không phải khẳng định mức tăng cố định trên mọi máy. [1][2]

4. Chi phí tính cost và đoạn nhận xét ghép báo cáo

4.1. Giải thích theo code hiện tại

`step_cost(board, next_state)` chỉ gọi `get_occupied_cells()`, đọc một hoặc hai phần tử `board.grid`, đếm ô mã 2 rồi cộng mức phạt. Nó không duyệt toàn bản đồ, không tính số nước đi tiếp theo và không gọi tìm kiếm. Với khối có tối đa hai ô, số thao tác của một lần tính cost là O(1) theo kích thước bàn. Trong UCS, hàm được gọi cho mỗi successor hợp lệ trước khi kiểm tra có cải thiện best_g hay không. [3][4]

Số lần tính cost tăng theo số cạnh hợp lệ được xét, không chỉ theo số node mở rộng. Raw CSV không ghi số lần gọi cost hay tổng successor, nên không thể từ số node suy ra thời gian riêng của cost. UCS còn sinh successor, tra bảng trạng thái, cập nhật parent/best_g, thao tác heap và xử lý entry lỗi thời. Phép đo có bật tracemalloc, nên thời gian là thời gian của solver trong cấu hình đo này. [4]

A* dùng cùng hàm cost nhưng còn tiền xử lý bảng split switch và tính heuristic khi cải thiện đường đi. Code tiền xử lý dựng các cặp switch và thực hiện các vòng thư giãn; mỗi lần heuristic còn có thể duyệt bảng switch. Với L16 có 64 công tắc tách, những phần việc này là cơ chế có thể góp phần làm A* chậm hơn dù mở ít node hơn. Đây là suy luận từ code, chưa phải phân rã thời gian đã đo; CSV hiện tại không đủ để định lượng phần trăm thời gian của heuristic hoặc cost. [1][4]

Profiler bao quanh hàm core nên gồm tiền xử lý heuristic của A*, nhưng không gồm khởi tạo worker/board ở bên ngoài core, giao tiếp tiến trình, replay kiểm chứng hoặc animation. Timeout 120 giây là giới hạn bên ngoài worker và khác với `search_time_ms`. Chất lượng đường đi (cost, độ dài) phải được nhận xét riêng với tài nguyên tìm kiếm (thời gian, bộ nhớ, node). [4]

4.2. Đoạn có thể ghép vào mục 6.2 của Report

Trên bộ 25 màn, mỗi thuật toán chạy ba lần, UCS tìm được lời giải có cost không cao hơn BFS ở mọi màn; UCS giảm cost tại L16, L24 và L25. Hai màn L24/L25 thể hiện rõ khác biệt giữa tối ưu số bước và tối ưu chi phí: BFS trả lời giải 16 hành động với path_cost 23.0, trong khi UCS trả 20 hành động với path_cost 20.0. UCS tăng bốn hành động nhưng giảm 13.04% cost theo policy phạt tiếp xúc fragile. Kết quả này không đồng nghĩa đường đi có xác suất rơi thấp hơn, vì engine áp dụng luật xác định và cost không mô hình hóa xác suất.

Đổi lại, UCS có trung vị thời gian lớn hơn BFS trên 24/25 màn và bộ nhớ đỉnh lớn hơn trên cả 25 màn. A* trả cùng cost và độ dài với UCS trên toàn bộ bộ đo, đồng thời mở ít node hơn trên 24 màn. Tuy nhiên, A* chỉ nhanh hơn UCS trên 20 màn và dùng ít bộ nhớ hơn trên 20 màn. Ở L16, A* mở 65,951 node so với 68,095 của UCS nhưng mất 35,061.705 ms so với 19,903.749 ms. Vì vậy, ít node hơn không bảo đảm thời gian thấp hơn. Chi phí tiền xử lý và đánh giá heuristic là một cách giải thích phù hợp với code, nhưng chưa được đo riêng. Các kết luận này giới hạn trong bộ dữ liệu và môi trường hiện tại; việc A* khớp UCS trên 25 màn không thay thế chứng minh tính admissible của heuristic. [1][2][3][4]

5. Rà kết luận UCS/cost trong các bản thảo

5.1. Report của Nam: mục 2.3, 4.3, 5.2 và 6.2

Mục 2.3 còn câu gợi ý "ví dụ: g(n) = 1/bước"; mục 4.3 và 6.2 vẫn là chỗ giữ chỗ. Vì vậy, bản Report hiện chưa có kết luận UCS thực nghiệm để xác nhận đã khớp. [5]

Nội dung thay cho gợi ý mục 2.3: "Mỗi cạnh hợp lệ có chi phí c(s,a,s') = 1 + 0.5 × k(s'), trong đó k đếm ô fragile bị chiếm ở trạng thái đến; g(s') = g(s) + c(s,a,s'). SPACE chơi tay không là cạnh tìm kiếm. BFS/DFS đếm số hành động; khi so cost giữa các thuật toán, dùng path_cost tính lại theo cùng công thức."

Nội dung điền mục 4.3 và 6.2: dùng mô tả/giả mã trong `ucs.md` (P02) và đoạn tại mục 4.2 của tài liệu này. Ở mục 5.2, dùng ms, KiB đỉnh cấp phát Python, lượt sinh successor, số hành động và path_cost; không gọi `memory_kb` là tổng RAM. Bảng P05 là nguồn cho mục 6.1.

5.2. Video-outline của Nam: phần III.3 và IV

Phần III.3 còn "Màn 31" và "path_cost tối ưu (18.0)"; bộ hiện tại chỉ có L1-L25. Thay cảnh UCS bằng L24 hoặc L25 và lời dẫn: "BFS đi 16 hành động, cost 23.0; UCS đi 20 hành động, cost 20.0. UCS chọn tổng cost thấp hơn, dù cần thêm bốn hành động." [1][6]

Phần IV còn ghi UCS/A* ở L4 có 17 bước, cost 57.0. Thay toàn bộ hàng so sánh cùng lúc, tránh trộn số cũ với số mới:

- BFS: 16 hành động; path_cost 16.5; 1,789 node; 357.309 ms; 543.01 KiB.
- DFS: 38 hành động; path_cost 38.5; 124 node; 12.834 ms; 62.60 KiB.
- UCS: 16 hành động; path_cost 16.5; 1,903 node; 500.618 ms; 705.98 KiB.
- A*: 16 hành động; path_cost 16.5; 1,245 node; 354.592 ms; 601.08 KiB.

Đây là trung vị của bộ benchmark, không phải số sẽ xuất hiện y hệt trong một lần quay GUI. Cụm "trọng số an toàn" nên diễn đạt thành "mức phạt tiếp xúc fragile theo policy"; không suy ra giảm xác suất thua. [1][3][6]

5.3. Astar-results của Thiện: bảng và các kết luận tổng quát

Bảng còn dùng Stage 32 và các số cũ ở Stage 1/12. Dùng khóa `(level_id, solver)` để tra CSV, không gọi "Dòng 12 CSV" là màn 12: một màn có bốn dòng thuật toán. Ví dụ L12 hiện là Bridge Maze, UCS/A* đều 28 hành động, cost 28.0; các chỉ số khác nằm ở mục 3. L1 hiện có UCS/A* lần lượt 55/18 node và 5.485/2.163 ms. [1][7]

Thay câu "chứng minh thực nghiệm … trong bất kỳ kịch bản nào" bằng: "Trên 25 màn đã đo, A* khớp cost của UCS; đây là kiểm chứng thực nghiệm hữu hạn. Bảo đảm tối ưu tổng quát còn phụ thuộc tính admissible của heuristic và cách cài đặt tìm kiếm."

Thay kết luận "A* luôn tiết kiệm … RAM … trên mọi kịch bản" bằng: "A* có đỉnh cấp phát Python thấp hơn UCS trên 20/25 màn, cao hơn ở L7, L8, L11, L21 và L22. A* nhanh hơn trên 20/25 màn, chậm hơn ở L5, L8, L10, L11 và L16." Không mô tả frontier "hình nón" như hình dạng đã được đo; CSV không ghi kích thước frontier theo thời gian. [1][2][7]

5.4. Review-notes của Thiện: trạng thái đối chiếu

Dòng policy `step_cost = 1 + 0.5 * fragile` phù hợp code nếu "fragile" được định nghĩa là số ô fragile bị chiếm ở trạng thái đến, không phải số ô trên cả bản đồ. Tuy nhiên, mục "Số liệu Benchmark" đang đánh dấu đã khớp và dẫn Stage 31/32; trạng thái này cần sửa theo bộ 25 màn và các câu thay ở trên. Mục goal test khi lấy khỏi frontier vẫn đúng với UCS/A*. [3][4][8]

6. Nguồn, tái tạo và sử dụng AI

1. handoff/quan/benchmarks/raw.csv, summary.csv, runs/*.json, environment.json, run-config.json; handoff/quan/level-groups.csv. Khóa nguồn là level_id/solver, thêm repeat_index cho lượt thô. Môi trường ghi trong metadata: Intel Core i5-12450HX, RAM 24 GB, Windows 11, Python 3.14.4; timeout 120 giây.
2. handoff/phuc/ucs-results-data.json, ucs-results-selected.csv và analyze_ucs.py; gói P05 gồm result-tables.*, chart-notes.*, charts/, provenance.json.
3. Src/solvers/costs.py; handoff/phuc/cost-policy.md và ucs.md (P01/P02).
4. Src/core/board.py; Src/solvers/member3_bfs.py, member5_ucs.py, member4_astar.py, base.py; Src/benchmarks/cli.py và README.md.
5. handoff/nam/Report.docx, các mục 2.3, 4.3, 5.2, 6.1 và 6.2.
6. handoff/nam/video-outline.docx, các phần III.3 và IV.
7. handoff/thien/astar-results.docx, bảng kết quả, giải thích cost/time/memory và đoạn nhận xét ghép báo cáo.
8. handoff/thien/review-notes.docx, phần đối chiếu policy cost, goal test và benchmark.

Từ gốc repo, tái tạo dữ liệu phân tích và tài liệu:

```
python handoff/phuc/analyze_ucs.py
powershell -ExecutionPolicy Bypass `
  -File handoff/phuc/build-documents.ps1 -Documents ucs-results
```

Nguồn soạn là ucs-results.md. Pandoc chuyển Markdown sang DOCX; bước sau đọc lại DOCX và dùng XeLaTeX xuất PDF. Lệnh phân tích kiểm tra 300 lượt/JSON, các trung vị, hash trùng P05 và tính lại các so sánh, không tạo số đo mới. Hash các bản thảo được rà nằm trong ucs-results-data.json.

Khai báo AI: Codex hỗ trợ đọc mã nguồn, trích nội dung DOCX, viết công cụ đối chiếu CSV, tính chênh lệch, soạn nhận xét/câu sửa và chuyển đổi, kiểm tra tài liệu. Số liệu gốc thuộc bộ đo của Quân; không dùng số minh họa thay dữ liệu. Phân tích mô tả và các suy luận từ code được phân biệt trong nội dung. Tác vụ đã yêu cầu: "làm P03"; quy trình soạn đã yêu cầu: tạo Markdown trước, chuyển qua DOCX bằng Pandoc rồi dùng XeLaTeX tạo PDF để kiểm tra. Phản hồi của Nam/Quân/Thiện sau bàn giao được ghi riêng, không được suy ra từ việc có file này.